ANALYSIS REPORT

# Harmonic Regression

Feedforward Transmission Error Baseline

Styled PDF edition generated from the canonical Markdown analysis report for improved readability, section hierarchy, and table presentation.

## Overview

This unified guide explains the repository's `Harmonic Regression` architecture from both a learning-oriented and technical perspective.

The purpose of the guide is to show how TE can be modeled with an explicit periodic basis instead of a generic dense neural network.

The reader should come away with an understanding of:

- why periodic structure matters for TE;
- how a truncated harmonic expansion is built;
- how the coefficients are learned or conditioned;
- why the model is much more interpretable than a plain MLP;
- how the implementation fits into the shared training pipeline.

  This is the first explicitly TE-structured model in the repository.

## Model Description

`Harmonic Regression` is a structured regression model.

Its defining idea is that the TE signal can be written as a finite harmonic series over angular position.

The model does not try to discover periodicity implicitly. It puts periodicity into the functional form from the beginning.

In the repository, the current implementation supports two modes:

- `static`

  A single coefficient vector is learned for the whole dataset.
- `linear_conditioned`

  The coefficients are adjusted by a linear projection of the operating-condition inputs. This makes the model a structured baseline with a clear physical interpretation.

## Operating Principle

The model assumes that the main TE shape can be approximated by a truncated harmonic series:

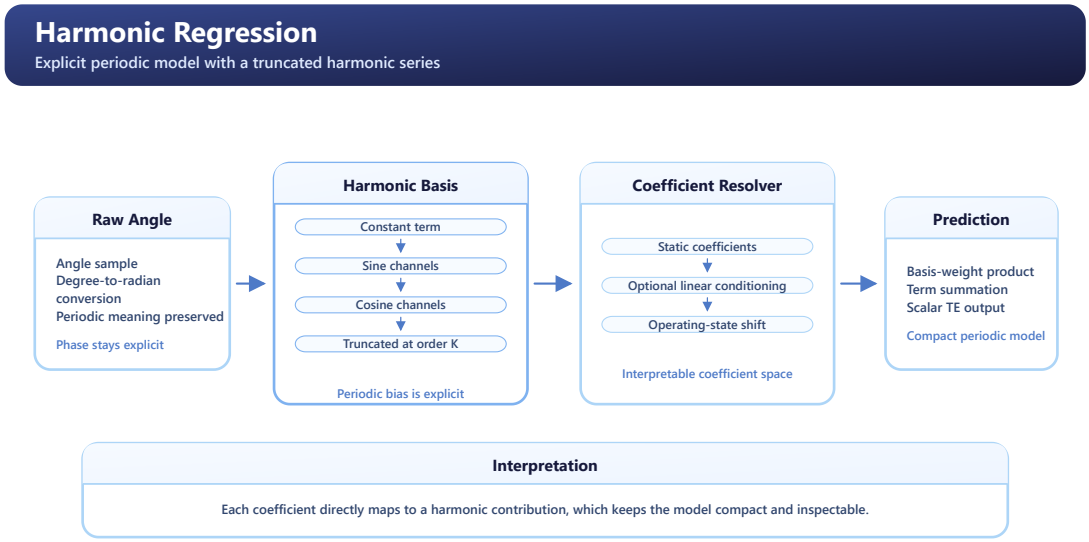
`TE(theta) = a0 + sum_k [a_k sin(k theta) + b_k cos(k theta)]`

### Operationally

1. the angle is converted from degrees to radians;
2. harmonic basis functions are generated up to the configured order;
3. the coefficient vector is resolved;
4. the basis and coefficients are combined into the final prediction.

In `static` mode, one coefficient vector is shared by all samples. In `linear_conditioned` mode, the operating conditions shift the coefficients in a linear way. This is a stronger inductive bias than the feedforward baseline because the model declares periodicity up front.

## Conceptual Map



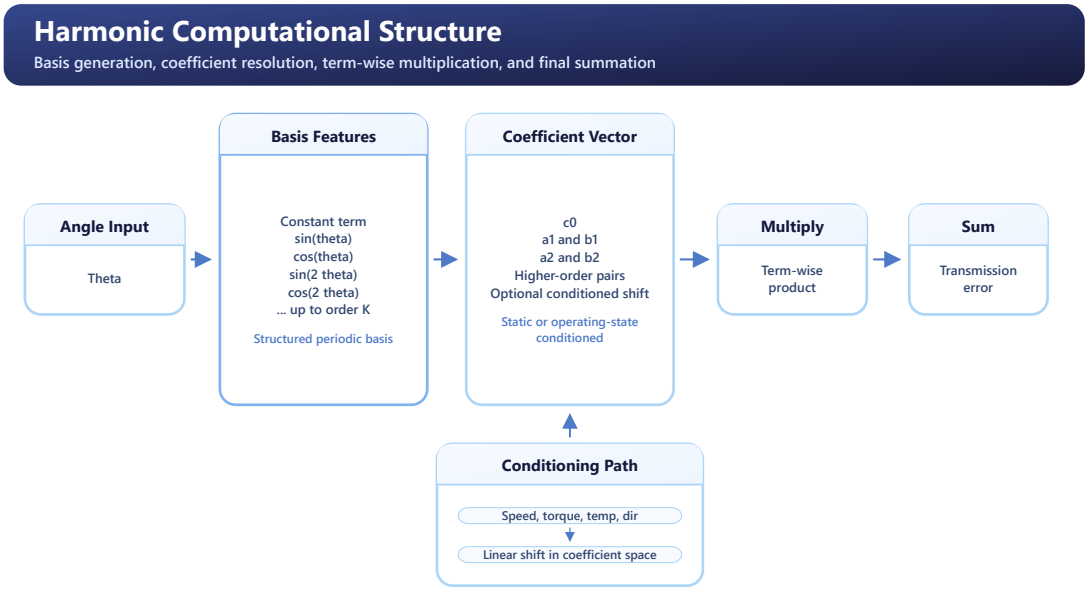

Harmonic Regression conceptual diagram

### The conceptual reading is

- raw angle becomes sine and cosine harmonics;
- the coefficient vector defines the signal shape;
- the final prediction is a weighted harmonic sum;
- the conditioned variant allows operating variables to adjust the coefficients.

The model can also be summarized as: ``` `text Input Point -> raw angle theta -> [1, sin(theta), cos(theta), ..., sin(K theta), cos(K theta)] -> harmonic coefficient vector -> element-wise product -> sum -> scalar TE prediction` Conditioned variant: `text [speed, torque, temperature, direction] -> linear projection -> coefficient correction -> base coefficients + correction -> harmonic prediction` ``` This diagram is useful because it shows that the model is closer to analytical regression than to a generic neural network.

## Architecture Diagram




Harmonic Regression architecture diagram

The architecture view emphasizes the computation graph:

- angle input;
- harmonic feature generation;
- coefficient resolution;
- term-wise multiplication;
- summation to a scalar TE output.

That is why the model is easier to interpret than the MLP baseline.

## Why This Model Exists

This model exists because TE is periodic with respect to angular position.

If periodicity is a strong part of the problem, it is better to encode it explicitly than to hope a generic network will rediscover it.

### The model therefore serves as

- a structured baseline;
- a physically readable reference;
- a bridge between pure regression and more hybrid models.

## Advantages

- Strong periodic inductive bias.
- High interpretability through harmonic coefficients.
- Compact parameterization.
- Easier to reason about than a dense neural baseline.
- Good fit for the TE domain where angular periodicity is central.

## Disadvantages

- Limited expressiveness outside harmonic structure.
- Can underfit nonlinear residual effects.
- Condition dependence is only linear in the current conditioned variant.
- Truncation order must be chosen carefully.
- Less flexible than a neural network when the data has richer interactions.

## Expected Behavior In The TE Context

This model is expected to work well when:

- TE is dominated by periodic components;
- the harmonic order is sufficient;
- operating-condition dependence is moderate or smoothly varying.

It may struggle when:

- residual nonlinear behavior is strong;
- condition dependence is highly nonlinear;
- the TE signal contains effects that are not well represented by the chosen harmonic truncation.

## Repository Implementation

The implementation is centered on these files:

- `scripts/models/harmonic_regression.py`
- `scripts/models/model_factory.py`
- `scripts/training/train_feedforward_network.py`
- `scripts/training/transmission_error_regression_module.py`

**`scripts/models/harmonic_regression.py`**

Key elements

- `HarmonicRegression.__init__(...)`

  Validates the TE input layout, stores the harmonic order, computes the feature count, and initializes:

  - `base_coefficient_tensor`
  - optional `conditioning_projection`
- `build_harmonic_feature_tensor(...)`

  Converts angle to radians and constructs the harmonic basis.
- `resolve_coefficient_tensor(...)`

  Produces the effective coefficients for either `static` or `linear_conditioned` mode.
- `forward_with_input_context(...)`

  Runs the full harmonic computation using both raw angle and normalized operating-condition features.
  This file is the mathematical center of the model.

**`scripts/models/model_factory.py`**

The model is registered under `model_type == "harmonic_regression"`.

That registration makes the architecture usable from configuration files and campaign scripts.

**`scripts/training/transmission_error_regression_module.py`**

This module is important because harmonic regression needs both:

- raw angle values for basis construction;
- normalized condition features for coefficient conditioning.

  The training module detects the contextual forward path and passes the correct inputs to the backbone.

**`scripts/training/train_feedforward_network.py`**

The outer training workflow is the same as for the feedforward baseline.

The difference is inside the backbone

- the trainer still manages validation, checkpointing, and testing;
- the harmonic model changes the way the prediction is computed.

  This makes the model easy to compare fairly against the plain MLP.

## Training Workflow

The workflow is the shared structured-neural pipeline:

1. point-wise TE samples are built;
2. train-split normalization is computed;
3. the harmonic model is instantiated from YAML;
4. the regression module forwards raw angle plus normalized context;
5. optimization proceeds with the shared loss and metrics;
6. the best checkpoint is reloaded;
7. validation and test results are reported.

## Training Logic In This Repository

### `TransmissionErrorRegressionModule.forward_regression_model(...)`

This is the key adaptation point for `harmonic_regression`.

The function checks whether the backbone implements:

- `compute_auxiliary_output_dictionary(...)`, or
- `forward_with_input_context(...)`

  Since `HarmonicRegression` provides `forward_with_input_context(...)`, the regression module passes both:

- the raw input tensor;
- the normalized input tensor.

  This is necessary because:

- the angular harmonic basis must be generated from the raw angle in degrees;
- the operating-condition channels are better handled in normalized form.

### `train_feedforward_network(...)`

This function is still the main trainer for this model family.

Nothing in the outer training loop is harmonic-specific. The specialization happens inside the backbone and the regression module interface.

This is a strong architectural choice in the repository:

- specialized model behavior;
- shared training infrastructure.

## Practical Interpretation

This model should be read as the question:

Can the TE signal be explained directly with a periodic series and a small amount of optional conditioning?

If the answer is yes, the model is attractive because it is compact and interpretable. If the answer is no, the residual or periodic-feature architectures become more relevant.

## Summary

`Harmonic Regression` is the first explicitly structured TE architecture in the repository.

It is not just a baseline. It is the clearest representation of the fact that TE is periodic in angular position, while still fitting cleanly inside the shared training infrastructure.